

Office Log System — Build Plan

One local web application with Postal Log + Incoming Invoices modules

1. Overview

One local web application with a single login and two modules:

- **Postal Log** (■■■■■ ■■ ■■■■■) — outgoing packages
- **Incoming Invoices** (■■■■■ ■■ ■■■■■■ ■■■■■■■■) — received invoices

After login the user sees two big buttons and chooses which system to use.

2. Goals

- Keep the same simple tech stack as the current postal system
- One login for both modules
- Shared partners list
- Clean separation of data (packages vs invoices)
- Match real paper workflow as closely as possible

3. Tech Stack

- **Backend:** Python 3 + Flask
- **Database:** SQLite (single file)
- **Frontend:** Vanilla HTML / CSS / JS (SPA)
- No build step, runs fully offline
- Bilingual (Macedonian / English)
- Dual theme (light / dark)

4. High-level Structure

```
office-log-system/
■■■ app.py                # Main Flask app + auth + shared routes
■■■ postal/
■   ■■■ routes.py        # Postal-specific API
■■■ invoices/
■   ■■■ routes.py        # Invoice-specific API
■■■ static/
■   ■■■ index.html       # Frontend (landing + both modules)
■■■ requirements.txt
■■■ start.bat
■■■ office.db             # SQLite database
```

5. Database Design

5.1 users

id	INTEGER PRIMARY KEY
username	TEXT UNIQUE NOT NULL
password_hash	TEXT NOT NULL
role	TEXT NOT NULL DEFAULT 'user' -- admin user
created_at	TEXT DEFAULT CURRENT_TIMESTAMP

5.2 partners (shared)

id	INTEGER PRIMARY KEY
name	TEXT NOT NULL
address	TEXT
city	TEXT
notes	TEXT
created_at	TEXT DEFAULT CURRENT_TIMESTAMP

5.3 packages (Postal module)

id	INTEGER PRIMARY KEY	
ref_number	TEXT NOT NULL	-- manual: 03-145, 08-12...
send_date	TEXT NOT NULL	
partner_id	INTEGER	-- optional FK → partners
recipient_name	TEXT NOT NULL	
recipient_address	TEXT	
city	TEXT	
package_type	TEXT NOT NULL	
year	INTEGER NOT NULL	
notes	TEXT	
created_at	TEXT DEFAULT CURRENT_TIMESTAMP	

5.4 invoices (Invoices module)

id	INTEGER PRIMARY KEY	
internal_number	TEXT NOT NULL	-- auto: 05-1, 05-2... (yearly reset)
invoice_number	TEXT	-- supplier's number (optional)
invoice_date	TEXT	-- date written on the invoice
received_date	TEXT NOT NULL	-- auto-filled with today
due_date	TEXT	
partner_id	INTEGER	-- FK → partners
bank_account	TEXT	-- 903, 787, 785M6, 785L1...
amount	REAL	
currency	TEXT DEFAULT 'MKD'	
payment_status	TEXT DEFAULT 'pending'	-- pending / paid / cancelled
notes	TEXT	
year	INTEGER NOT NULL	
created_at	TEXT DEFAULT CURRENT_TIMESTAMP	

6. Numbering Rules

Invoices

- Format: **05-1, 05-2, 05-3...**
- Resets every year on 1 January → starts again at 05-1
- System automatically suggests the next number for the current year
- received_date is automatically set to today when creating a new entry

Postal Packages

- ref_number stays **manual**
- User types the full number (examples: 03-145, 08-12, 01-3...)
- Common prefixes: 03- (most used), 08-, 01-, 02-, 04-

7. User Interface Flow

1. Login screen
2. Landing page — two big buttons:  / Postal Log and  / Incoming Invoices
3. Each module has: List/Table view, Add/Edit modal, Search + filters, Export (CSV/Excel)
4. Shared elements: Language switcher, Theme switcher, User menu

8. Main Features by Module

Postal Log

- Add / Edit / Delete packages
- Manual ref_number (■■■■■■■■■■ ■■■■)
- Search by recipient + filter by year/date
- Table view + Group by recipient view
- Export CSV / Excel
- Package types (same as current system)

Incoming Invoices

- Add / Edit / Delete invoices
- Auto internal_number (05-X) with yearly reset
- Auto received_date = today
- Fields: invoice date, due date, partner, bank account, amount, status
- Search + filters + Export CSV / Excel
- Simple payment status (pending / paid)

Shared

- Partners management (add / edit partners)
- User management (admin only)
- Bilingual UI + Light / Dark theme

9. Implementation Phases

Phase 1 — Foundation

- Project structure
- Database tables (users, partners, packages, invoices)
- Login / logout / session
- Landing page with two big buttons
- Basic routing between modules

Phase 2 — Postal Module

- Migrate existing postal functionality
- Keep current package types and manual ref_number
- Link to partners (optional)
- Export

Phase 3 — Invoices Module

- Invoice CRUD
- Auto internal_number (05-X) with yearly reset
- Auto received_date
- Bank account field + basic payment status
- Export

Phase 4 — Shared & Polish

- Partners management UI

- Consistent design between both modules
- Yearly reset logic tested
- Final testing

10. Notes & Decisions

- One SQLite database file for simplicity
- created_at is kept as automatic timestamp
- received_date on invoices is auto-filled (can be edited if needed)
- Postal ref_number remains fully manual
- Partners are shared between both modules
- System runs locally (and can be accessed on LAN)

11. Future Ideas (not in v1)

- Automatic next-number suggestion also for postal prefixes
- Dashboard with counts / pending invoices
- Attachments (scan of invoice)
- Better reporting

Plan created: 2026-10-02